Exhibition demo guide — Windows VM, no Docker

One elevated PowerShell window, the URDS console in the browser, and one guided script that runs the whole sequence from a file being saved to an attacker being stopped and the files coming back.

Every command below is short on purpose: long lines copied out of a PDF pick up line breaks and spaces and stop working. Type them, or copy them one at a time.

0. What the jury will see

The PowerShell window runs the guided script and narrates each step with ✔ / ✘ results. The projector shows the URDS console at **http://localhost:8501**: the system's own operator dashboard, the same view an administrator would use, updating as the demo runs.

Page	What it shows
Overview	service health, files watched, alerts, processes terminated, ledger blocks and chain integrity; the latest alert traced through Detected → Classified → Recorded → Responded; entropy of recent events; recent events and ledger blocks
Events	every file event the Monitor holds, filterable (alerts, benign, governed, deleted, verdict, path); one event in full: entropy, container check, hash, model verdict, ledger block, attributed process, response result, governance decision
Ledger	chain integrity and verification time; every block, paged and filterable by type and file, with a per-block link check; the block inspector with full hashes and event data; CSV and JSON export
Response	monitor status with start/stop, process attribution status, every response action, suspension leases, snapshots and restores, the whitelist
Model	accuracy, precision, recall, F1 and ROC AUC per model; score any file on demand (the result is logged to the ledger)

Which page to show at each step is in section 3.

The 8 steps (about 8 minutes in total):

#	Step	Test cases
1	Normal work is left alone: a text file, a CSV and a real ZIP, no alarm	TC-03
2	A disguised encrypted file is caught: ZIP header over ciphertext	TC-02, TC-06
3	Live attack: the attacker process is identified and terminated	TC-01, TC-07, TC-09
4	Ransomware simulator vs URDS: 20 documents, how many were lost, then undo	TC-01
5	Recovery from a real Windows shadow copy, hash-verified against the ledger	TC-04
6	Tamper-evident log: edit the database, URDS names the edited block	TC-05
7	Access control: 401 without a token, 200 with, refusal logged	TC-10
8	The numbers: precision/recall/F1, latency, kill time, CPU, memory	TC-06, TC-08

1. Fresh VM: setup is two short lines

VM: Windows 10 or 11, 64-bit, 2+ CPU cores, 4 GB+ RAM, 10 GB free disk, internet access. Nothing needs to be installed first: not Git, not Python, not winget.

1. Start menu → type `PowerShell` → right-click → **Run as administrator**. Use this window for the whole demo.
2. Type (or paste) these two lines, pressing Enter after each:

```
$u = "https://raw.githubusercontent.com/VSshashank/unified-ransomware-system"
irm "$u/main/scripts/bootstrap_vm.ps1" | iex
```

It downloads the code to `C:\urds` and runs `scripts\setup_vm.ps1`, which:

Step	What happens
1/7	finds Python 3.13 (or 3.12); if there is none it installs 3.13 via winget, or straight from python.org when winget is missing or broken
2/7	builds `.venv` (rebuilds it if it was made with the wrong Python)
3/7	installs all six services' packages, the slow part (5–10 min)
4/7	trains the ML model if `models\` is missing (about 1–3 min)
5/7	adds Windows Defender exclusions for the folder and the Python processes
6/7	turns on the file-audit rule that lets URDS name, and stop, the attacking process
7/7	starts all six services, prints UP for each, starts the Monitor and opens the console

It ends with **READY**. Re-running it is safe; finished steps are skipped.

To rehearse a branch before it is merged into `main`, set the branch first, in the same window:

```
$env:URDS_BRANCH = "claude/exhibition-demo-setup-3b9d28"
$u = "https://raw.githubusercontent.com/VSshashank/unified-ransomware-system"
irm "$u/$env:URDS_BRANCH/scripts/bootstrap_vm.ps1" | iex
```

Offline VM: copy the repository folder (including `models\`) to `C:\urds`, install Python 3.13 by hand, then `cd C:\urds` and run `.\scripts\setup_vm.ps1 -SkipInstall -Wheels D:\wheels` with a wheelhouse built on a connected Windows machine (`pip download -r <each requirements.txt> -d wheels`).

2. Before the jury arrives (2 minutes)

In the elevated window, from `C:\urds`:

```
cd C:\urds
.\scripts\start_stack.ps1
```

All six lines must say `UP`, and the last lines must say **Attribution LIVE**. If it says *Attribution NOT available*, detection, the AI and the ledger still work, but no attacker will be killed in steps 3 and 4. See section 6.

Then:

1. In the browser that opened (the console, on Overview), press **F11** for full screen.

2. Clean the stage: `.\scripts\exhibition_demo.ps1 --reset`
3. Optional dry run without the slow steps: `.\scripts\exhibition_demo.ps1 --auto --steps 1,2,7`

If PowerShell says *running scripts is disabled on this system*, run this once in that window:

```
Set-ExecutionPolicy -Scope Process Bypass -Force
```

3. Running the demo

Presenter mode: you press Enter, the screen follows:

```
.\scripts\exhibition_demo.ps1
```

At every prompt: **Enter** = go on, **s** + Enter = skip this step, **q** + Enter = stop.

Kiosk mode: runs by itself in a loop, for when you are not at the stand:

```
.\scripts\exhibition_demo.ps1 --auto --loop --pause 10
```

Other options:

Option	Use
`--steps 2,4,6`	run only these steps (a 3-minute version)
`--family slowburn`	gentler simulator for step 4 (fewer files lost); default `locker`
`--delay-ms 200`	faster simulator, so more files get hit before the kill
`--skip-vss`	skip step 5 if shadow copies do not work on this VM
`--verbose`	show step 3's full technical transcript

What to show and say at each step

Step	Console page	Look at
1 Normal work	Overview	Latest event: the ZIP at entropy 8.0, *Benign (valid container)*, no ledger block
2 Disguised file	Overview	Latest alert: container *forged*, Classified *Ransomware*, Recorded *Block #n*
3 Live attack	Overview, then Events	Responded *Terminated*; on Events, the alert's process and PID
4 Simulator	Events (Alerts)	one alert per encrypted document, each with its response
5 Recovery	Response	Recovery table: *snapshot_created*, *file_recovered*, integrity verified
6 Tamper-proof log	Ledger	Chain integrity turns *Broken* and names the block; the inspector shows it failing; then *Valid* again
7 Access control	Ledger	filter type *auth_failure*: the refused request is a block
8 Numbers	Model	per-model accuracy, precision, recall, F1, ROC AUC

1. **Normal work.** "URDS watches every file change. It measures how random the bytes are, on a scale of 0 to 8, and checks the file's inner structure. This ZIP scores 8.0, as random as encrypted data, but its structure is genuine, so there is no alarm. Without that check, every archive would be an alert."
2. **Disguised file.** "This file says it is a ZIP, but behind the header there is only ciphertext. The Monitor sees a forged structure, the AI model confirms ransomware, and the evidence is sealed in the ledger. Everything

turns red in under a second."

3. **Live attack.** "Now a separate program behaves like ransomware and keeps running. URDS reads the Windows security log to find exactly which process wrote the file, and terminates it in well under two seconds. Note the PID on screen."
4. **Simulator.** "A harmless simulator starts encrypting twenty documents. Each alert on screen is one encrypted document. URDS stops it after a few files, and then every file is put back." *(Say honestly: the faster the attack, the more files are hit before the kill; see docs\VM_RETEST_ADDENDUM.md.)*
5. **Recovery.** "We take a Windows shadow copy, the file is encrypted, and URDS restores it. It then checks the restored file's SHA-256 fingerprint against the one recorded in the ledger before the attack."
6. **Tamper-proof log.** "Each record carries the fingerprint of the one before it. Here an attacker edits a record directly in the database to hide the alert. The chain breaks and URDS names the exact block. Every block is here in full: its hash, the hash of the one before it, and what it records." *(Honest limit: an attacker who recomputes every later hash is not caught; `scripts\tamper_sweep.py` : 20/20 in-place tampers detected, 0/8 structural.)*
7. **Access control.** "No token, or a forged one: refused. A valid signed token: allowed. And the refusal itself is written to the ledger."
8. **Numbers.** "Precision 96%, recall 95%, detection in about 10 ms, the kill in under 0.1 s, about 1% CPU."

4. Between visitors

```
.\scripts\exhibition_demo.ps1 --reset
```

This restores any simulator files, empties `watched_files` and empties the Monitor's whitelist. The ledger keeps every block: it is append-only by design. Full restart if anything looks wrong:

```
.\scripts\stop_stack.ps1
.\scripts\start_stack.ps1
```

5. Before the exhibition: run the test suites once

Keep the output in a spare window; do not run these on stage.

```
cd C:\urds\services\gateway;  ..\..\.venv\Scripts\python.exe -m pytest -q
cd C:\urds\services\ledger;   ..\..\.venv\Scripts\python.exe -m pytest -q
cd C:\urds\services\monitor;  ..\..\.venv\Scripts\python.exe -m pytest -q
cd C:\urds\services\response; ..\..\.venv\Scripts\python.exe -m pytest -q
cd C:\urds\services\dashboard; ..\..\.venv\Scripts\python.exe -m pytest -q
cd C:\urds
```

Expected counts from the last recorded run: gateway 108, ledger 99, monitor 660, response 206 (2 permanent skips on Windows). TC-11 (simultaneous attacks) is shown by the monitor suite. A load-sensitive test may fail on a busy VM: re-run it alone.

6. If something breaks

Symptom	Fix
running scripts is disabled on this system	`Set-ExecutionPolicy -Scope Process Bypass -Force`, then retry

Symptom	Fix
Not elevated	close the window, reopen PowerShell with **Run as administrator**
Setup stops at 1/7 (Python)	install Python 3.13 64-bit from python.org (tick *Add to PATH*), then `.\scripts\setup_vm.ps1 -SkipInstall`
Setup stops at 3/7 (pip)	check the VM's internet; re-run `.\scripts\setup_vm.ps1 -SkipInstall`. It resumes
A service says NOT UP	the last log lines are printed below it; full log: `Get-Content run\logs\<service>.err.log -Tail 30`
Demo preflight: *AI model not loaded*	`.venv\Scripts\python.exe src\train_behavioral_model.py`, then `.\scripts\start_stack.ps1`
Attacker tracking is OFF	`.\scripts\setup_vm.ps1 -SkipInstall -NoStart` in an elevated window, then `.\scripts\start_stack.ps1`
Simulator not stopped in step 4	same as above: attacker tracking must be LIVE
Step 5: *shadow copy failed*	VSS is off on this VM: run with `--skip-vss` and say so; detection is unaffected
Console blank / "Gateway data unavailable"	`.\scripts\start_stack.ps1` (it restarts everything cleanly)
Port already in use	`.\scripts\stop_stack.ps1`, then start again
Simulator files vanish	Defender quarantine: re-run `.\scripts\setup_vm.ps1 -SkipInstall -NoStart` to re-add the exclusions
401 in your own `Invoke-RestMethod` calls	token expired (2 h): `. .\scripts\start_watching.ps1` refreshes `$t` and `$h`

7. What to say when asked what is not done

- Network isolation is a dry run unless `RESPONSE_ISOLATION_ENABLED=true`.
- The ledger does not defend against an attacker who rewrites the whole chain.
- The faster simulator families can exceed the "under 5 files" bound (`docs\VM_RETEST_ADDENDUM.md`).
- TC-12 (blockchain) is Semester 2.

Appendix A — manual commands (if the guided script cannot be used)

Run from `C:\urds` after `. .\scripts\start_watching.ps1`.

```
$py = ".venv\Scripts\python.exe"
$w = "$PWD\watched_files"
& $py scripts\attack_chain_demo.py --watch-host $w --watch-container $w
& $py scripts\ransomware_simulator.py --target-dir $w\tc01 --family locker
& $py scripts\ransomware_simulator.py --target-dir $w\tc01 --restore
& $py scripts\verify_vss.py --volume C:\
& $py scripts\si_demo.py --db run\data\ledger.db
curl.exe -i http://localhost:8000/ledger/entries
curl.exe -i -H "Authorization: Bearer $t" http://localhost:8000/ledger/entries
```

Empty the whitelist (the old guide used `POST`; the API only accepts `PUT`):

```
$body = '{"paths":[],"hashes":[]}'
$url = "http://localhost:8001/monitor/whitelist"
irm -Method Put $url -ContentType "application/json" -Body $body
```

Appendix B — what was wrong with the previous guide

Problem on a fresh VM	Fixed by
`setup_vm.ps1`, `start_stack.ps1`, `start_watching.ps1`, `stop_stack.ps1` were not in the repository, so step 2 failed right after the clone	all four are now in `scripts\`
Command 1 needed winget + Git, which many fresh VM images lack or have broken	`bootstrap_vm.ps1` downloads the GitHub ZIP; no Git or winget needed
Long commands broke when copied from the PDF (`$en v:Path`, `path lib`)	one short bootstrap line; the demo steps are a script, not pasted one-liners
Dot-sourcing failed: *running scripts is disabled* (Windows' default policy)	bootstrap sets the policy for its window; setup sets `RemoteSigned` for the user
The Microsoft Store `python.exe` stub counted as "Python installed", so `python -m venv` failed	setup ignores the stub and looks for a real 3.13/3.12
Any other Python version broke pip: numpy 2.5.1 has no 3.11 wheel, pandas 2.2.3 none for 3.14	setup requires and installs exactly 3.13 (3.12 also works)
Under PS 5.1, `$ErrorActionPreference = "Stop"` turned pip's stderr warnings into fatal errors	native tools are checked by exit code instead
`start_watching.ps1` got HTTP 400: the gateway requires `file_patterns`	the request now sends `file_patterns: []`
`Test-NetConnection` took seconds per check; the wait loop looked hung	plain HTTP health checks, with the failing service's log printed
Streamlit on `0.0.0.0` raised a Windows Firewall dialog mid-demo	dashboard binds `127.0.0.1` (use `-Lan` to expose it)
Defender's behaviour monitor could kill the simulator before URDS did	process exclusions for the venv and base Python
Whitelist reset used `POST` → HTTP 405	`PUT` (Appendix A), and `--reset` does it for you